TECHNICAL POST-MORTEM AND IMPLEMENTATION GUIDE

Why This AlphaZero Loop Worked

Lessons from repeated failures, a clean-room rebuild, and a successful 8x8 Breakthrough run

For students implementing AlphaZero-style systems
26 August 2026

Project-declared success. The accepted iteration-26 network beat the already-trained phase-6 starting model 39–1, then beat iteration 15 by 36–4, and finally beat the depth-4 alpha-beta baseline 49–1 in a 50-game paired-opening arena. The 50-hour training program was still running when this post-mortem was written.

The lesson is not that one hyperparameter was magic. The system improved when its conventions, tests, data, search, optimization, exploration, and evaluation finally formed one coherent feedback loop.

Contents

How to read this post-mortem

Reinforcement-learning success invites retrospective storytelling: after the score improves, every design choice can look inevitable. This guide therefore separates three kinds of statement.

Evidence class	Meaning	Example in this project
Invariant	A property established by unit, differential, or metamorphic tests.	Action round trips, perspective conversion, sign-free backup, and exact make/unmake restoration.
Measured result	A comparison from a completed run with recorded settings.	The 1,000-game versus 10,000-game pretraining match and paired checkpoint arenas.
Causal hypothesis	A plausible explanation consistent with the evidence, but not isolated by a full ablation.	Why the combined serious-training recipe escaped the feedback failures of earlier attempts.

Causality warning. The successful 8x8 run changed several compute-efficiency and training choices together. Its success proves that the complete recipe works in this setting. It does not prove that every ingredient is necessary or that each contributed independently. Where the evidence does not support a causal claim, this guide says so.

The guide in one sentence. First make every boundary testable; then give the learner enough diverse, search-improved data; finally measure strength with paired games rather than with training loss.

1 What succeeded

The project succeeded in the operational sense that matters for an AlphaZero assignment: repeated

$$\textsc{Play} \longrightarrow \textsc{Search} \longrightarrow \textsc{Train}$$

updates produced agents demonstrably stronger than their trained predecessors and stronger than a fixed classical baseline. That is much stronger evidence than a falling loss, a plausible-looking policy, or a single entertaining game.

Checkpoint comparison	Score	Protocol
Phase-6 iteration 5 vs. raw pretrained initializer	52–8	60 paired 0.1-second games.
Serious iteration 15 vs. phase-6 iteration 5	39–1	20 six-ply openings, colors reversed, fixed 64-simulation search.
Serious iteration 26 vs. serious iteration 15	36–4	The same scheduled strength-check protocol.
Serious iteration 26 vs. depth-4 alpha-beta	49–1	25 six-ply openings, colors reversed, 0.1 seconds per move.

The final alpha-beta match was balanced by color: iteration 26 scored 24–1 as Player 1 and 25–0 as Player 2. All 50 games completed. Its Wilson interval was 89.5%–99.6%. The match ran on an L40S, whereas the earlier 2–58 phase-6 result used RTX 2080-class hardware. The exact magnitude of improvement is therefore not hardware-controlled. The conclusion that iteration 26 beat the baseline under the measured protocol is nevertheless unambiguous.

What success does not establish

- It does not establish optimal play, or even the strongest feasible Breakthrough agent.
- It does not prove that the full 50-hour budget was necessary; the agent was already strong by iteration 26.
- It does not isolate the individual contribution of playout-cap randomization, replay size, root noise, search growth, or learning rate.
- It does not make validation loss a strength metric. The decisive evidence is the checkpoint ladder and the external baseline match.
- It does not imply that the same numeric settings transfer unchanged to games with different branching factors, horizons, rewards, or symmetries.

2 Why earlier attempts failed

The earlier projects did not fail because they lacked sophistication. They failed because individually reasonable components disagreed at their boundaries. A process-local augmentation counter reset. An inference ensemble required by training was omitted. Rare teacher samples dominated batches. Optimizer and checkpoint lifecycles became unclear. Some arenas compared identical model hashes or counted time forfeits as strength.

AlphaZero is unusually unforgiving of such defects because each model creates the next model's data. A small systematic error is not merely observed; it is fed back into the system.

Earlier pattern	Observed failure	Successful-project response
Complex absolute-player encoding plus four transformations	A prior 8x8 continuous run lost augmentation state between short learner processes and omitted the required inference ensemble. The actor developed asymmetric blind spots and material gifts.	Use two mover-relative planes. Convert value perspective in one wrapper. Player swap becomes a test, not training augmentation; random left-right reflection is enough.
Tiny or badly weighted learning data	One historical diagnosis gave rare teacher positions about 71 times the per-position weight of neural positions. A nominal batch of 256 had effective size near 18, and every trained epoch worsened validation.	Use a recent FIFO of homogeneous neural self-play for reinforcement learning. Size training by presentations per fresh position, not opaque source weights.
Optimizer and actor lifecycle spread across jobs	Restarted learner processes and rollback machinery made it easy to lose Adam state or compare one checkpoint under two labels.	Save Keras optimizer state with every checkpoint. The newest trained network always generates the next tranche; reference checkpoints are diagnostic only.
Repeated live seeds	Supposedly fresh runs reproduced the same first games. Deterministic evaluation risked measuring one repeated trajectory.	Do not fix seeds in production self-play, replay sampling, or arenas. Generate distinct random opening prefixes, then reverse colors within each pair.
Too little search mistaken for a batching bug	At 8/2 simulations, sequential and batched self-play both produced 0–32 Player-1 results.	Separate algorithm quality from throughput. A 64-simulation diagnostic restored a healthy 19–13 split; the serious schedule began at 128/16.
Metrics and gates accumulated faster than understanding	Binary tactical gates and actor-versus-candidate machinery sometimes rejected useful models or evaluated identical hashes.	Keep direct invariant tests, a continuous tactical score, compact metrics, and standalone paired arenas.

Most important comparison. The historical failures came from sophistication crossing untested boundaries: process-local state, replay weights, time-control grace, checkpoint identity, symmetry assumptions, and actor lifecycle. The successful rebuild deliberately reduced the number of boundaries and made the remaining ones observable.

3 The contracts that made the system coherent

3.1 One public value convention

Outside the neural network, every value is an *absolute Player-1 value*: $+1$ favors Player 1 and -1 favors Player 2. The network sees the mover as its own side, so its raw value is mover-relative. A single neural-boundary wrapper performs the conversion:

$$v_{\text{abs}}(s) = p(s)\, v_{\text{rel}}(s), \qquad z_{\text{rel}}(s) = p(s)\, z_{\text{abs}},$$

where $p(s) = +1$ when Player 1 is to move and $p(s) = -1$ when Player 2 is to move.

The tree stores absolute values. Backup therefore never flips a sign. Selection is the only turn-dependent operation:

$$a^* = \arg\max_a \left[p(s)\, Q(s,a) + c_{\text{puct}}\, P(s,a) \frac{\sqrt{\sum_b N(s,b)}}{1 + N(s,a)} \right].$$

Player 1 selects larger absolute Q; Player 2 selects smaller absolute Q. Tests exercise both players on hand-computed trees.

Tradeoff. Absolute tree values are not inherently better than mover-relative tree values. Mover-relative values can be elegant, but then every edge crossing changes perspective and backup usually flips a sign. Absolute values made the teaching implementation easier to audit because one wrapper and one exploit term contain all perspective logic. The portable rule is not to always use absolute values. It is to choose one convention, write it down, and test every boundary against it.

3.2 A trusted ladder below self-play

The project did not ask a neural reinforcement-learning loop to diagnose its own foundations. Each layer had a cheaper oracle:

- **Rules:** an independently written legal-move generator and exact make/unmake restoration.
- **Terminal logic:** goal-row, no-reply, capture, and the rule that a terminal move does not switch player.
- **Action encoding:** encode/decode round trips for every legal move in random games.
- **Symmetry:** metamorphic tests for legal moves, policy indices, outcomes, and reflected positions.
- **Search:** uniform-prior or rollout search that finds immediate wins for both players before a neural network is introduced.
- **Network boundary:** solver-labelled supervised positions that establish that policy masking, value perspective, fitting, save/load, and inference all agree.

This ladder matters because a self-play loop that did not improve is an extremely low-information symptom. A reflected position that changes its terminal outcome, or a Player-2 node that prefers the larger absolute Q, points almost directly to a faulty line of code.

3.3 Canonical state and symmetry

The neural input contains two planes: pieces belonging to the mover and pieces belonging to the opponent. The board is oriented so the mover always advances in the same neural direction. This removes an irrelevant color distinction without discarding game information.

For Breakthrough, player swap is already represented by mover-relative canonicalization, so applying it again during training adds no new example. Left-right reflection is the genuine geometric symmetry. The replay buffer stores the original position; a random reflection is applied after sampling, with the policy target transformed by the same action permutation.

> **Common symmetry bug.** Transforming the board but not the policy indices silently assigns probability to the wrong actions. Test a one-hot policy through every transformation and its inverse. Also test legal-action masks and terminal rewards. A visual inspection of transformed boards is not enough.

3.4 Exact state restoration

MCTS applies and retracts many moves. A one-bit mutation leak can corrupt distant branches while leaving the root position apparently plausible. For random legal positions and every legal action, the project checks:

$$\texttt{snapshot(state)} = \texttt{snapshot(unmake(make(state, action)))}.$$

The snapshot includes the board, player to move, terminal fields, and any cached legal-action information. For a new environment, hash the complete state before and after every debug search until this contract is beyond doubt.

4 Data and optimization choices that mattered

4.1 The 5x5 ladder was an instrument, not a toy

The smaller board shortened games, reduced the branching factor, and made exact or strong tactical labels affordable. It exposed value-perspective errors, BatchNorm behavior, memorization, label imbalance, and weaknesses in the first tactical suite before expensive 8x8 jobs were submitted.

The project did *not* stretch a padded 5x5 network into an 8x8 network and call that evidence about native 8x8 strength. Native board sizes received native models and native data. What transferred was the tested process.

> **Generalization.** Choose the smallest faithful instance of the target environment: a smaller board, shorter horizon, reduced action set, or deterministic scenario. It must preserve the contracts you are testing. If the reduced problem changes the meaning of an action, observation, or reward, it is a different environment, not a debugging ladder.

4.2 Pretraining gave self-play a competent starting distribution

The 8x8 initializer was trained on 10,000 rollout-PUCT games containing 421,561 stored positions. A 1,000-game subset was not nearly as good: it lost 24–76 to the 10,000-game model. The large corpus therefore bought measurable strength, not merely a prettier loss curve.

The stored corpus contained real positions. Symmetry was sampled during training rather than materialized as duplicate records. This keeps storage honest and allows a fresh random transformation on every presentation.

Pretraining is not a requirement of AlphaZero in principle. In a modest compute budget it is an engineering choice: it avoids spending the first expensive neural self-play hours rediscovering elementary legality and tactics. For another environment, a teacher may be a rollout search, alpha-beta, heuristic controller, solver, demonstration set, or a mixture. The target is not imitation forever; it is a policy/value pair good enough for search to produce useful improvements.

4.3 Architecture followed evidence

The final network was intentionally modest: 48 filters, three residual blocks, and a direct three-channel 1×1 convolutional policy head. Batch normalization was removed after supervised diagnostics showed that its moving statistics and small, correlated batches complicated the learning signal.

This is a project-specific result, not a universal ban on BatchNorm. Larger, well-shuffled batches or different normalization schemes may work well. The portable lesson is to ask the network to pass a controlled supervised task before blaming self-play. If a small model cannot fit and generalize on exact labels, more actors will only generate expensive ambiguity.

4.4 Replay exposure was explicit

The serious loop used a FIFO replay of roughly 25,000 recent neural positions, batch size 128, and about two training presentations per fresh position. Adam state was retained, with learning rate 2.5×10^{-4}.

The useful quantity is not epochs in isolation. It is how often each fresh position is expected to influence an update:

$$\text{reuse ratio} = \frac{\text{training examples drawn in an iteration}}{\text{new positions added in that iteration}}.$$

Too little reuse underfits expensive search targets. Too much reuse memorizes a small, highly correlated window and can erase older competencies. The right window also depends on policy drift: recent data are on-policy but narrow; older data improve diversity but were generated by weaker policies.

> **Do not fix divergence by freezing learning. Lowering the learning rate to nearly zero can make losses and weights look stable while stopping improvement. First diagnose perspective, labels, effective sample size, replay composition, normalization, optimizer state, and search quality. Stability without learning is not success.**

4.5 The loss was conventional; the targets were the hard part

For canonical state s, search policy π, and mover-relative outcome z, the learner minimized

$$\mathcal{L}(\theta) = (z - v_\theta(s))^2 - \pi^\mathsf{T} \log p_\theta(\cdot \mid s) + \lambda \|\theta\|_2^2.$$

Illegal logits are masked before a policy is used. Search visit counts are normalized over legal root actions to create π. The value target is the eventual game result, converted exactly once at the neural boundary.

Most catastrophic loss problems were not caused by this equation. They came from feeding the equation targets expressed in the wrong coordinate system, perspective, action indexing, or sample distribution.

5 A serious but simple AlphaZero loop

5.1 The loop

The production design remained conceptually small:

```
while time_is_left:
    games = play_parallel_games(current_model)
    replay.add(games)

    train_current_model(replay)
    save_model_and_optimizer()

    if strength_check_is_due:
        play_paired_match(current_model, reference_model)
```

There is no candidate-acceptance gate in the learning path. The latest trained model generates the next tranche. Scheduled matches report progress; they do not roll the actor backward. This avoids a noisy gate freezing learning or creating a two-model lifecycle that students cannot audit.

5.2 Parallel search changed throughput, not semantics

Actors advanced 32 games together and batched neural leaf evaluations on the GPU. Within one game, PUCT selection, expansion, backup, and root targets kept their ordinary meaning. Batching was treated as a scheduling optimization rather than a new search algorithm.

This distinction paid off during diagnosis. A severe Player-1 collapse appeared at an 8/2 simulation budget in both sequential and batched implementations. Raising the budget to 64 simulations restored a 19–13 split. The defect was insufficient search, not batching.

Portable principle. Build and test a slow reference search first. The optimized actor should agree with it on legal moves, terminal values, visit accounting, and fixed-network decisions. Benchmark speed only after semantic equivalence is established.

5.3 Playout-cap randomization spent search where labels mattered

Not every self-play move was stored. A minority of positions received the full search budget and became training targets; intervening moves used a smaller budget to advance games cheaply. In the serious schedule, about 25% of turns were full-search targets.

This trades target density for target quality and game throughput. It works when the cheap turns remain plausible enough to reach useful states and the full-search positions cover the game. Log target ply, player, outcome, and game phase. If full searches cluster accidentally, the replay buffer will inherit that blind spot.

5.4 Search capacity grew with the model

The schedule increased full/fast simulations from roughly 128/16 to 192/24 and then 256/32. Early networks do not justify the same search expense as later ones, but extremely shallow search can create self-confirming nonsense.

A portable calibration procedure is:

1. On a fixed checkpoint, sweep several search budgets.
2. Measure throughput, color balance, tactical score, policy entropy, and head-to-head strength.
3. Choose the smallest budget before the strength curve clearly flattens.
4. Repeat after the network becomes materially stronger.

5.5 Exploration in self-play; controlled diversity in evaluation

Self-play added Dirichlet noise to root priors and sampled from visit counts during the first 12 plies. Later moves were selected more sharply. This creates varied openings while allowing games to become decisive.

Evaluation removed root noise and stochastic move sampling. Diversity came from independently generated six-ply opening prefixes. Each opening was played twice with colors reversed:

$$\text{random opening} + \text{deterministic play} + \text{color reversal}.$$

This protocol samples many positions without allowing move noise to dominate the strength estimate. The pair controls much of opening and first-player advantage. It is reproducible if the opening list is saved, without reusing one live random seed everywhere.

5.6 Strength checks measured what losses could not

Every five hours, the loop played a 40-game paired-opening match against a fixed reference. The run continued for the full budget even if several checks were flat; the ladder was evidence, not a stopping rule.

Use validation loss to diagnose training distribution, overfitting, and serialization. Use games to measure playing strength. These quantities can move in opposite directions because:

- policy targets are produced by changing searches;
- replay is non-stationary and correlated;
- stronger play may visit harder positions;
- cross-entropy rewards matching visit distributions, not winning by itself;
- value calibration and move ranking can improve differently.

6 Bug prevention: make contracts executable

6.1 Tests worth writing before the first long run

1. **Reference rules.** Compare the optimized legal-action generator with a literal, independently written version on thousands of random states.
2. **Transition round trip.** For every legal action in sampled states, make then unmake and compare complete snapshots.
3. **Action round trip.** Require $\text{decode}(\text{encode}(a)) = a$ for every legal action.
4. **Terminal semantics.** Hand-construct each terminal cause and check both the winner and whether the player-to-move field changes.
5. **Perspective.** Evaluate color-swapped or mover-swapped copies and verify the exact expected relation between relative and absolute value.
6. **Symmetry.** Transform state, legal actions, policy, and outcome; invert the transform; require equality.
7. **Hand-computed PUCT.** Build tiny trees for both players and check the chosen child, visits, W, and Q after each backup.
8. **Neural masking.** Assert exactly zero probability on illegal actions and unit probability mass over legal actions.
9. **Serialization.** Save and reload a trained model; compare predictions and optimizer iteration/state, not just weights.
10. **Actor identity.** Hash the model used to create every tranche and the model written after training. A completed update must change the expected hash.

6.2 Symptom-to-test debugging table

Symptom	Likely classes of cause	Fastest discriminating tests
One player collapses	Value sign, terminal turn switch, directional encoding, asymmetric search budget, missing symmetry.	Hand PUCT tests for both players; color-swapped tactical pairs; sequential versus batched search at a larger budget.

Symptom	Likely classes of cause	Fastest discriminating tests
Policy loss falls but play does not improve	Targets too weak, illegal-mask mismatch, replay overuse, identical actor, evaluation too noisy.	Hash checkpoints; inspect root targets; play fixed-position search sweeps; run a paired deterministic arena.
Value is accurate on training but wrong in games	Perspective conversion, leakage between train/validation, correlated split, normalization-state mismatch.	Split by complete game; evaluate swapped copies; compare training and inference modes; reload before evaluation.
Good tactics, terrible long games	Replay lacks strategic phases, search too shallow, fast-turn policy drifts, terminal reward sparse.	Histogram target plies and phases; label sampled states with a stronger search; increase budget on the same checkpoint.
Two fresh evaluations are identical	Reused seeds, reused opening file, deterministic actor launch, identical checkpoint hashes.	Log opening hashes, first moves, process seeds, and model hashes before the match.
Arena result is extreme or unstable	Color/opening bias, time forfeits, unequal hardware, too few independent openings, failures counted as losses.	Reverse colors, report failures separately, compare clocks and hardware, and use confidence intervals over opening pairs.
Batched search disagrees with reference search	Virtual-loss accounting, duplicate expansion, stale priors, wrong backup order, state mutation.	Use a deterministic mock network; compare per-edge visits after each batch; hash state before and after search.
Training suddenly stabilizes after restart	Optimizer state lost, learning rate reset or frozen, replay path changed.	Record optimizer iteration, learning rate, replay count, and a fixed prediction vector before and after restart.

6.3 Log enough to reconstruct a failure

For every training iteration, record:

- actor checkpoint path and cryptographic hash;
- optimizer iteration and learning rate;
- new games, new positions, replay size, and training presentations;
- full/fast search budgets and fraction of stored turns;
- root-noise parameters and move-temperature schedule;
- outcomes by player, game length, target ply, policy entropy, and search value;
- policy loss, value loss, finite-value checks, and gradient or weight norms if available;
- arena opening IDs, colors, failures, clocks, hardware, and result.

Do not turn logging into a second framework. A CSV row and a short text summary per iteration are sufficient if field names and units are stable.

6.4 Use continuous diagnostics, not brittle tactical gates

The final tactical suite contained 20 exact, balanced positions rather than six hand-picked puzzles. Its value score was continuous:

$$\frac{1}{20}\sum_{i=1}^{20} v_{\mathrm{abs}}(s_i)\, z_{\mathrm{abs}}(s_i).$$

Positive values mean the prediction tends to agree with the exact outcome; magnitude reflects confidence. Color-swapped copies test a perspective invariant; they are not additional independent evidence.

A tactical suite is a probe, not a complete strength measure. Keep the raw per-position outputs so one easy family cannot hide a systematic failure.

7 Adapting the method to another environment

7.1 Decide the value object before writing MCTS

For a two-player zero-sum game, a scalar value is enough. Choose either a fixed global viewpoint or a mover-relative viewpoint and derive selection and backup algebra on paper.

For a general-sum or multiplayer game, a scalar current-player value can discard information. Store a value vector

$$\mathbf{v}(s) = (v_1(s), \ldots, v_k(s))$$

and let the parent choose using the component belonging to its decision maker. Chance nodes use expectations rather than maximization. In partially observable games, the network input and tree state must represent an information state; tests must explicitly prevent hidden-state leakage.

7.2 Canonicalize only exact equivalences

Environment property	Good adaptation	Required test
Players are interchangeable under a known transform	Canonicalize to the mover or a fixed seat.	Transform state, actions, and rewards; invert and recover the original sample.
Board has geometric symmetries	Sample a group element during training or ensemble at inference if justified.	Legal actions and transition results commute with every transform.
Rules contain asymmetric terrain, roles, or objectives	Keep the asymmetry in the observation; do not augment it away.	Construct a counterexample position where the proposed transform would change the game.
Stochastic transitions	Represent chance outcomes explicitly or train on sampled returns.	Check probability mass, reproducibility under controlled RNG, and unbiased backup estimates.
Very long horizon	Use denser diagnostics, search-budget scheduling, and possibly reward/value normalization.	Test terminal rewards separately from any shaped reward and inspect phase coverage in replay.
Continuous actions	Replace full policy enumeration with a suitable proposal/search method.	Test bounds, density normalization, sampling, and action-transform Jacobians if needed.

7.3 Match data generation to the environment's difficulty

The successful recipe used rollout-PUCT pretraining because Breakthrough had a cheap legal simulator and meaningful shallow tactics. Other environments may need:

- demonstrations to reach rare rewarding states;
- curriculum levels to control horizon or branching factor;
- an exact solver for small states;
- a heuristic or model-predictive controller as teacher;
- prioritized scenario generation for safety-critical or rare events.

Whichever source is chosen, measure the resulting policy, state coverage, outcome balance, and effective sampling weights. One million rows says little if they contain 10,000 copies of a few trajectories.

7.4 Adapt evaluation to the source of variance

Paired color-reversed openings are natural for deterministic, alternating, two-player board games. In another environment, pair or stratify over the largest nuisance variables:

- random seeds and initial conditions in stochastic control;
- maps, scenarios, and traffic patterns in simulators;
- roles or teams in asymmetric games;
- demand regimes and time periods in operations problems.

Keep policy stochasticity out of evaluation unless stochastic execution is part of the deployed agent. Report failures and timeouts separately from legitimate losses.

8 A staged implementation protocol

The following order minimizes the cost of being wrong. Each gate should have a written pass condition and a saved artifact or test result.

1. **Environment gate.** Implement the smallest faithful ruleset, a literal reference transition/legal-action function, terminal rewards, and state cloning or make/unmake. Pass differential and round-trip tests.
2. **Action and value gate.** Define action indexing, neural canonicalization, the public search value convention, and parent utility. Exhaust action round trips and player/symmetry metamorphisms.
3. **Non-neural search gate.** Use uniform legal priors and a rollout or exact evaluator. Find immediate wins for every player, preserve the input state, and verify hand-computed Q selection.
4. **Network gate.** Train on solver-labelled positions. Require held-out learning, legal masking, save/load equality, and perspective consistency.
5. **Teacher data gate.** Generate a reconstructable search corpus. Audit game uniqueness, seat balance, terminal legality, target mass, and validation split by whole episode.
6. **One-cycle gate.** From one checkpoint: generate, add to replay, train with preserved optimizer state, save, reload, and generate again. Assert the actor hash changed and all metrics are finite.
7. **Small-environment learning gate.** Run enough cycles to beat pretraining and fixed baselines in paired arenas. Diagnose regressions before scaling.
8. **Native target gate.** Create a native network and new data. Transfer only measured process choices. Calibrate search budget before serious compute.
9. **Serious-run gate.** Fix a compute budget, replay exposure, search schedule, exploration policy, strength-check protocol, recovery plan, and artifact paths before submission.

> **Rule for expensive jobs. Every expensive run should answer a question that a cheaper run cannot answer. If the next 20 GPU-hours are merely checking whether action indices are correct, the project has skipped a gate.**

9 The successful Breakthrough recipe

These settings describe the successful regime. The relationships are more portable than the exact numbers.

Component	Breakthrough setting	Portable principle
Neural input	Two mover-relative planes.	Canonicalize only under an exact, reversible transform for state, action, and reward.
Network	48 filters, 3 residual blocks, no BatchNorm.	Use the smallest model that passes supervised and save/load gates. Enlarge only after data and search are credible.
Pretraining	10,000 rollout-PUCT games; 421,561 stored positions.	Begin neural self-play with plausible search targets; measure a data-size curve rather than guessing corpus sufficiency.
Replay	Recent FIFO, about 25,000 neural positions.	Balance on-policy freshness against trajectory and skill diversity.
Optimization	Batch 128, reuse ratio about 2, Adam retained, learning rate 2.5×10^{-4}.	Track presentations per fresh position and preserve optimizer state.
Augmentation	Random left-right reflection after sampling.	Store real samples once; transform state and policy together during training.
Self-play exploration	Root Dirichlet noise and visit sampling for the first 12 plies.	Explore early enough to diversify trajectories, then sharpen to finish games.
Parallelism	32 games advanced together; neural leaves batched.	Batch inference without changing search semantics; compare with a slow reference implementation.
Search economy	25% full-search stored targets; fast intervening turns.	Spend computation on the positions that become labels; audit phase coverage.
Search schedule	128/16, then 192/24, then 256/32 full/fast simulations.	Calibrate the minimum useful budget on a fixed checkpoint and revisit as the model improves.
Evaluation	40 games every 5 hours; distinct six-ply openings, colors reversed, no search noise.	Use deterministic agents over diverse paired initial conditions. Keep training independent of a noisy acceptance gate.

9.1 Anti-patterns to avoid

- Do not change value perspective in several modules.
- Do not apply a symmetry to observations without applying its exact action permutation to policy targets.
- Do not infer corpus size from augmented presentations; count unique stored states and complete games.
- Do not call a random position split validation when adjacent states from the same game appear in both sets.
- Do not treat a near-zero learning rate as a cure for bad labels.
- Do not conclude that batching is wrong before repeating the test at a useful search budget.
- Do not let failed games, timeouts, or identical checkpoints enter an arena score silently.
- Do not reuse one fixed seed throughout production and mistake reproducibility for diversity.
- Do not scale to the target environment because a reduced architecture happens to accept padded inputs.
- Do not add gates, rollback rules, or distributed services faster than you can test their identities and state transitions.

10 Readiness checklist

Before launching	Evidence required
Rules and state	Differential legal-action tests; complete make/unmake snapshots; all terminal causes covered.
Action representation	Exhaustive legal encode/decode round trips; legal-mask agreement.
Perspective	One written convention; both-player hand PUCT tests; swapped-position metamorphic tests.
Symmetry	State/action/policy/reward transform tests and inverse recovery.
Network	Exact-label supervised learning; illegal probability zero; save/load prediction and optimizer equality.
Data	Unique game count, position count, outcome/seat balance, whole-game split, effective weights, and augmentation policy recorded.
Search	Immediate tactics for each player; state preserved; fixed-checkpoint budget sweep; reference versus batched agreement.
Training cycle	Actor hash, replay growth, finite losses, optimizer continuity, and changed post-update hash.
Evaluation	Distinct initial conditions, pairing/stratification, equal resources, failure count, model hashes, and uncertainty interval.
Recovery	Atomic checkpoint, replay location, iteration log, resume test, and known last complete unit of work.

11 What we would do next

Declaring success does not end the experiment. The remaining 50-hour run maps the learning curve: whether improvement continues, saturates, oscillates, or regresses as the replay distribution moves.

The most valuable follow-up is not immediately a larger network. It is a small ablation matrix around the successful recipe:

- hold the checkpoint fixed and compare search budgets;
- train matched short continuations with and without playout-cap randomization;
- compare two replay windows at equal fresh-position reuse;
- compare retained and reset optimizer state;
- repeat arenas across hardware with simulation-count rather than wall-clock limits.

These experiments turn causal hypotheses into measured claims. They also give students a more useful result than a claim that the final settings worked: a picture of which parts are robust and which depend on the environment and compute regime.

12 Conclusion

The successful project was not rescued by a mysterious neural-network trick. It became learnable when the entire feedback loop agreed with itself:

1. the environment produced correct states and rewards;
2. action, symmetry, and perspective transformations were explicit;
3. search used the value convention intended by training;
4. the starting data were large and competent enough to matter;

5. replay and optimization exposed fresh targets at a controlled rate;
6. parallelism accelerated inference without redefining search;
7. self-play explored, while evaluation controlled its sources of noise;
8. strength was measured by paired games and checkpoint identity was verified.

The transferable lesson. An AlphaZero implementation is a circular scientific instrument. Every arrow in the circle needs a unit, a viewpoint, an identity, and a test. Once those contracts are trustworthy, the sophisticated mathematics can do its job. Before that, more compute simply makes the wrong experiment run faster.

A Failure timeline and evidence

- A prior continuous 8x8 run was invalidated by symmetry-state loss across learner processes and by an omitted inference ensemble.
- A large-data diagnosis found rare teacher examples with roughly 71 times the per-position weight of neural examples and effective batches near 18 despite a nominal size of 256.
- A supposedly meaningful arena was shown invalid when both checkpoint hashes were identical; another was invalidated by time forfeits.
- The clean-room project established 5x5 correctness and learning before creating native 8x8 weights and data.
- The 1,000-game pretrained model lost 24–76 to the 10,000-game model, showing that corpus scale mattered in this regime.
- Phase-6 iteration 5 beat the raw pretrained initializer 52–8 but lost 2–58 to alpha-beta on RTX 2080-class hardware.
- Serious iteration 15 beat phase-6 iteration 5 by 39–1.
- Serious iteration 26 beat iteration 15 by 36–4.
- Serious iteration 26 beat depth-4 alpha-beta 49–1 on an L40S, using 25 distinct six-ply openings with colors reversed.

B Interpretive limits

The strongest evidence in this document is comparative, not absolute. The iteration ladder uses controlled paired openings. The final alpha-beta result is decisive for its recorded protocol, but its magnitude cannot be compared directly with the earlier phase-6 match because the hardware differed under a wall-clock time control.

The project did not run a factorial ablation over all serious-loop ingredients. Claims about why the bundle worked are therefore engineering judgments informed by failed runs, unit tests, diagnostics, and measured matches. They should guide the next experiment, not substitute for it.

Finally, Breakthrough is deterministic, perfect-information, alternating, two-player, and zero-sum. The value algebra, evaluation pairing, and canonical player transform exploit those facts. Section 7 describes the changes required when an environment does not share them.